AMPHORA COATING IP / AUDIT LOOP OPERATIONS

The Machine That Audits Us

2026-08-04 — how the hourly falsification loop actually behaves

209 fires / **78** verdicts / **43** repairs landed / **0** signing violations since the fix

54 refuted / 22 survived / 2 blocked / 18 modules attacked
33.6 agent-hours of compute over 208 hours of wall clock / 16% duty cycle

This is the operations report for the audit loop itself, not for the science it produces. It exists because a loop that grades its own homework is worth nothing: the sections below show what the loop got wrong, what that cost in agent-hours, and what was changed so it stops. Every figure is computed from the launcher's own append-only records by the scripts shipped alongside this PDF.

Master manifest SHA-256
f5623b3171d65de2dc805b740aef71ad6163554dcf9b5b394d84181b3ad6bf08

AUDIT-ONLY / NO PROMOTION AUTHORITY / COMPANION TO THE PORTFOLIO STATUS REPORT OF THE SAME DATE

SECTION 1

What the loop is, and what a verdict has to survive

Once an hour, a launcher script on xbabe1 wakes a single claude-sonnet-5 agent inside the mvp20-split worktree on xbabe2, hands it one job, and gives it the hour. The agent has no authority to promote a module, edit the campaign backlog, or touch the manifest. It can do exactly one thing that counts: produce a DSSE-signed evidence receipt and append one line to the append-only falsification ledger.

Over the 208 hours from Jul 26 23:00 to Aug 04 15:00 UTC the loop fired **209** times and banked **78** verdicts across **18** modules.

The asymmetry that makes the number mean something. The agent that writes a verdict is not the process that records it. The launcher re-derives the verdict from the filesystem after the agent exits, using three checks the agent cannot influence. A verdict the launcher cannot independently confirm is written down as `UNKNOWN` — the agent's own claimed outcome is preserved in `outcome_raw` but banks nothing, and the hour is counted as a loss against the loop. 12 of the 209 fires were downgraded this way.

The three mechanical checks

1. **Content-hash attribution against a pre-fire snapshot.** Before the agent starts, the launcher records the sha256 of every file under the module tree. Afterwards it diffs. Only files whose bytes changed *during this fire* can be claimed as this fire's artifacts, which is what stops an agent from re-declaring somebody else's receipt as its own work. The launcher's `protected_check` — did the fire touch a path it was not allowed to touch — returned **OK** on 81 of the 82 fires that ran it.
2. **Real Ed25519 DSSE verification against the pinned Phase-0 key.** Not a schema check, not a "`signature` field is present" check: the launcher rebuilds the PAE pre-authentication encoding and verifies the signature against the key in `config/phase0-trust-store.v1.json`. An envelope signed with a repo-local dev key fails here even though it is a structurally perfect DSSE object. `signing_check` returned **OK** on 81 of 82 fires (98.8%); the one `NOT_RUN` is the fire killed by a launcher bug, section 4(e).
3. **A fire-id-bound ledger line.** Since Aug 02 00:13 every fire carries a unique `fire_id` and the launcher checks that a ledger line carrying *that* id exists. This closes the gap where an agent did real work, signed it correctly, and then never recorded it — which happened, and is why `ledger_has_fire_id` is checked separately from the signature. It returned true on 60 of 64 fires (93.8%); 60 of the 78 ledger lines now carry a fire id.

Where the rule is not clean, stated up front

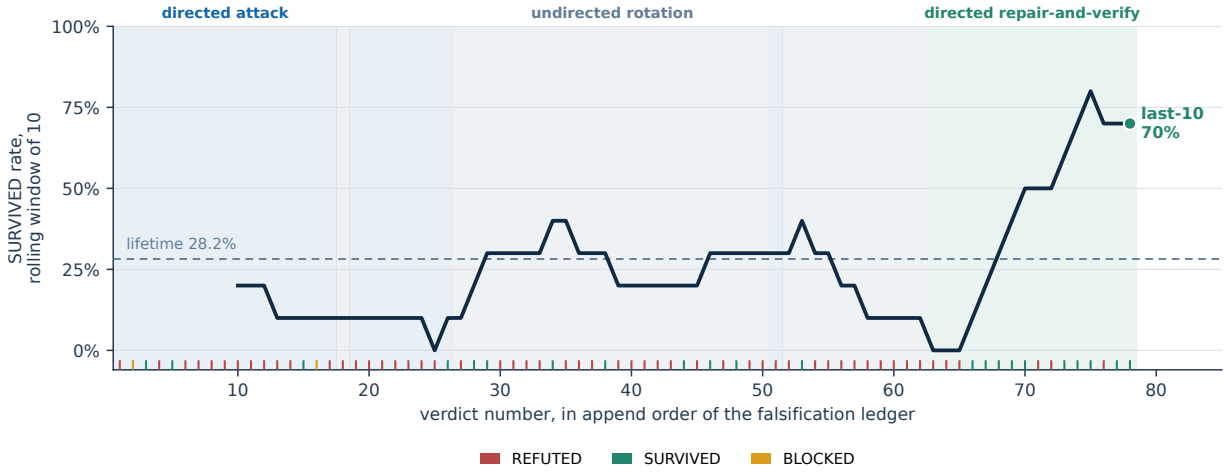
A downgrade denies the *fire* credit. It does not retract a line the agent already appended, because the ledger is append-only and rewriting it would cost more than the inconsistency does. Joining the ledger to the fire record on `fire_id` shows **7** of the 78 verdicts were written by fires the launcher then marked `UNKNOWN`. Those 7 verdicts are counted everywhere in this report, so the reader should know what they are: in all 7 of 7 cases the fire's own `signing_check` passed — a receipt that verifies against the pinned key exists — and the downgrade came from `verified_new_signed = 0`, the launcher being unable to attribute a *newly created* signed artifact to that hour. The science was recorded; the attribution was not. No ledger line in the file cites a receipt that fails signature verification.

What this report is not. None of this says the science is right. It says the bookkeeping is adversarial. A REFUTED verdict here means one agent, given an hour and a signing key, could not make a claim survive its own attack — nothing more. The value of the loop is that it makes that outcome cheap enough to happen 78 times in 208 hours.

SECTION 2

The convergence exhibit

This is the single most important page in the report. It plots the rolling-10 SURVIVED rate across all 78 verdicts in the order the ledger recorded them, with the campaign regime shaded behind.



Window	verdicts	survived	SURVIVED rate
Lifetime (all 78)	78	22	**28.2%**
First 58	58	12	20.7%
Last 20	20	10	**50.0%**
Last 10	10	7	**70.0%**

What changed, and why it is not the loop getting soft

The rise is not a drifting standard. It is a change of *job*. Split by campaign regime:

Regime	verdicts	refuted	survived	rate
Directed attack (waves C, W)	26	20	4	15.4%
Undirected rotation (backlog empty)	36	28	8	22.2%
Directed repair-and-verify (R, V, V1)	16	6	10	**62.5%**

For the first 62 verdicts the loop was hunting: pick a standing claim, try to break it. Claims written before anyone was attacking them broke at a high rate, which is the expected and correct result. From verdict 63 the backlog switched to repair-and-verify: an interactive session lands a repair, and a *later, independent* fire that did not write the repair is sent to attack it. Those verdicts survive more often because they are testing something that has already been fixed once — and, critically, 3 of the 13 verifies still came back REFUTED (section 8), so the check is doing work rather than rubber-stamping.

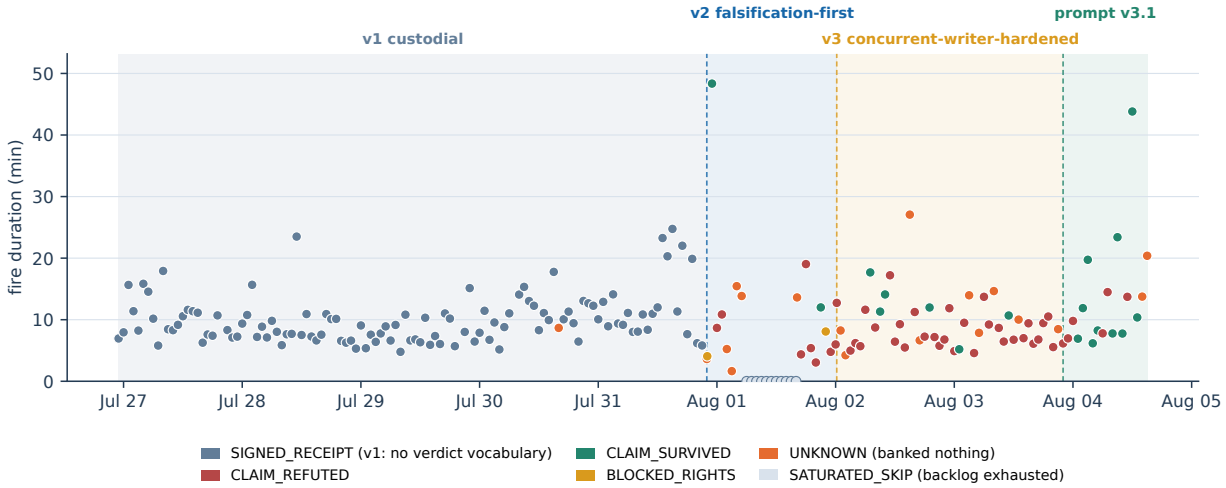
Small-sample caveat, stated plainly. A rolling window of 10 over 78 points is noisy, and the whole tail is one wave of work. A single verdict moves the last-10 line by 10 percentage points; the normal-approximation 95% interval on the last-10 figure of 70% is roughly ±28 points, which overlaps the lifetime rate. Read the 16-verdict repair-and-verify block as a regime with a plausibly higher survival rate, not as a measured trend with a slope. The honest claim is narrow: directed repair work survives independent attack more often than undirected attack on unrepaired claims did, and the loop is producing the evidence either way.

SECTION 3

Four generations of launcher, in 208 hours

The loop was rewritten three times while it was running. Each boundary below is *derived*, not asserted: it is the first fire that emitted a field the previous generation was structurally incapable of emitting.

- **v1 custodial** (from Jul 26 23:00). The launcher had exactly one success vocabulary word, `SIGNED_RECEIPT`. There was no way for an agent to record "this claim is wrong". All 115 outcomes in this era are `SIGNED_RECEIPT` and the ledger is empty of verdicts — the loop was accumulating evidence and calling all of it a success.
- **v2 falsification-first** (from Jul 31 21:57). Introduced `CLAIM_REFUTED` / `CLAIM_SURVIVED` / `BLOCKED_*`, the campaign backlog, the `signing_check` and `protected_check` gates, and `outcome_raw` so that a downgrade is visible rather than silent.
- **v3 concurrent-writer-hardened** (from Aug 02 00:13). Added the per-fire `fire_id` and the `ledger_has_fire_id` check, so a fire cannot claim credit for a ledger line another fire wrote, and the append-only ledger survives two writers in the same hour.
- **prompt v3.1** (from Aug 03 22:00, waves R/V/V1). Not a launcher change but a prompt change: mandate the pinned evaluator seed, require the agent to self-verify its own PAE encoding before exiting, require artifacts to be declared honestly, and forbid parking on a background wait. Signing violations go to zero here and stay there.



Launcher generation	n	signed	refut.	surv.	unkn.	skip	runtime mean/med (min)	mean sign.viol.	down-grades
v1 custodial	116	115	0	0	1	0	10.0 / 9.1	0.03	0
v2 falsification-first	29	0	8	2	6	11	10.4 / 7.0	0.07	5
v3 concurrent-writer-hardened	46	0	31	6	9	0	9.3 / 8.6	2.26	7
prompt v3.1 (waves R/V/V1)	18	0	6	10	2	0	13.3 / 10.1	0.00	0

Read the table down the *refuted* column. v1 produced 0 refutations in 116 fires because it could not express one. The moment the vocabulary existed, the same portfolio started refuting itself at scale. Nothing about the science changed on Jul 31 21:57 — only what the loop was allowed to write down.

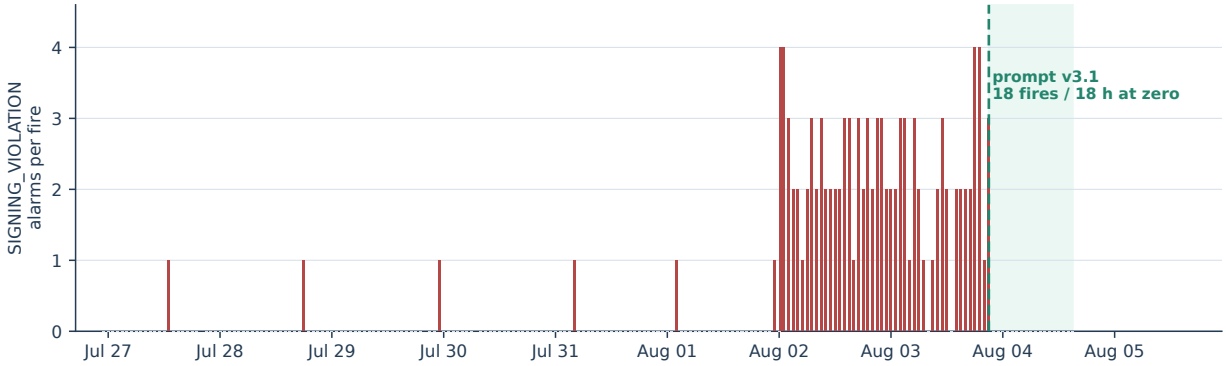
SECTION 4

Defects found in the loop itself

The loop's job is to falsify claims. Its own operating record is a claim. This section is what happened when the same standard was turned inward: five defects, all found from the launcher's own logs, all fixed, all costed.

(a) Signing discipline — agents signing with a key nobody trusts

Agents were signing evidence with the repo-local dev key instead of the pinned Phase-0 evaluator seed. The envelopes were structurally valid DSSE and verified fine against the key that produced them, which is exactly why this is dangerous: it looks signed. The launcher's Ed25519 check against `config/phase0-trust-store.v1.json` rejected them, the verdicts were downgraded to `UNKNOWN`, and real work was thrown away.



Across the run the launcher raised **110** `SIGNING_VIOLATION` alarms over 50 fires, 104 of them inside the v3 window alone. Prompt v3.1 mandated the pinned evaluator seed plus an agent-side PAE self-verify before exit. Since Aug 03 21:00 the count is **zero** for 18 consecutive fires spanning 18 hours.

(b) Artifact-declaration hygiene — most of those alarms were false

The first read of the alarm count was wrong, and correcting it matters more than the raw number. Agents were declaring plain-JSON diagnostics, scratch analyses and intermediate result files in the same `artifacts` list as their signed receipts. The launcher dutifully checked every declared artifact for a valid signature, found none on the diagnostics, and raised a violation per file.

The evidence that these were mis-declarations, not unsigned evidence. Of the 110 alarms, **106** were raised on fires whose *own* independent `signing_check` returned `OK` in the same hour — the fire's actual receipt verified against the pinned key while the alarm fired on a sibling diagnostic file. Only 4 alarms landed on a fire that failed its signing check outright. The fix was a prompt-level artifact-declaration rule: declare signed receipts as artifacts, list working files separately.

The residue of the real failures is visible in the wasted-fire accounting overleaf: 8 fires reached a genuine verdict, signed it with the wrong identity, and banked nothing.

SECTION 4, CONTINUED

(c) Background-wait stalls — fires that parked and expired

5 fires emitted no `RUN_SUMMARY` at all. The launcher has nothing to verify in that case, so the hour is a total loss: no receipt, no ledger line, no diagnostic. The failure mode is an agent that starts a long-running job in the background and then waits on a monitor until the hour expires, banking nothing it had already computed. Prompt v3.1 forbids the pattern outright. The affected fires: Jul 30 16:00, Aug 03 08:00, Aug 03 13:00, Aug 04 14:00, Aug 04 15:00.

Two of these are live, not historical. The two most recent stalls are the last two fires in the record (Aug 04 14:00 and Aug 04 15:00), both on campaign item `V1-KPI-QC` for process-qc-twin. That item has now consumed 2 of its 3 permitted fires and produced nothing. This is an open incident at the time of the pull, not a closed one.

(d) Ledger timestamp corruption — a literal backslash in an append-only file

The launcher prompt built each ledger line by interpolating `$(date)` through a nested-quoting layer. Twice, the escape leaked and a literal backslash was written in front of the timestamp:

ledger row	stored timestamp	module / item
55	`\2026-08-03T15:08:37+00:00`	ip-fto-guard / -
74	`\2026-08-04T10:07:20+00:00`	sourceability-ehs / V-A12b

Fixed by having `jq` generate the timestamp inside the JSON construction rather than passing a shell-expanded string through the prompt. The two damaged rows are **permanent**: the falsification ledger is append-only by design, and rewriting history to fix a cosmetic defect would destroy the property that makes the ledger worth reading. Both rows parse correctly once the leading backslash is stripped, which every script in this report's `scripts/` directory does, and which any downstream consumer must do too.

The defect register

The full set, including the two described overleaf. "Residue" means the generating defect is fixed but a permanent artefact of it remains in an append-only file; "monitor" means the guard exists and has never fired.

Defect	how it was caught	fix	status
(a) non-anchored signing key	launcher Ed25519 verify against the pinned trust store rejected the envelope	prompt v3.1: pinned evaluator seed + agent-side PAE self-verify	closed
(b) artifact misdeclaration	106 of 110 alarms fired on fires whose own `signing_check` passed	prompt v3.1 artifact-declaration rule	closed
(c) background-wait stall	5 fires emitted no `RUN_SUMMARY` and banked nothing	prompt v3.1: no background wait	2 live
(d) ledger timestamp escape	two `ts` values fail ISO-8601 parse	`jq` builds the timestamp inside the JSON	residue
(e) launcher `pipefail` abort	one fire produced work but no record at all	launcher v2.1	closed
(f) evidence schema flip	autocommit guard emits `RESTORED_SCHEMA_FLIP`	guard shipped, log under watch	monitor

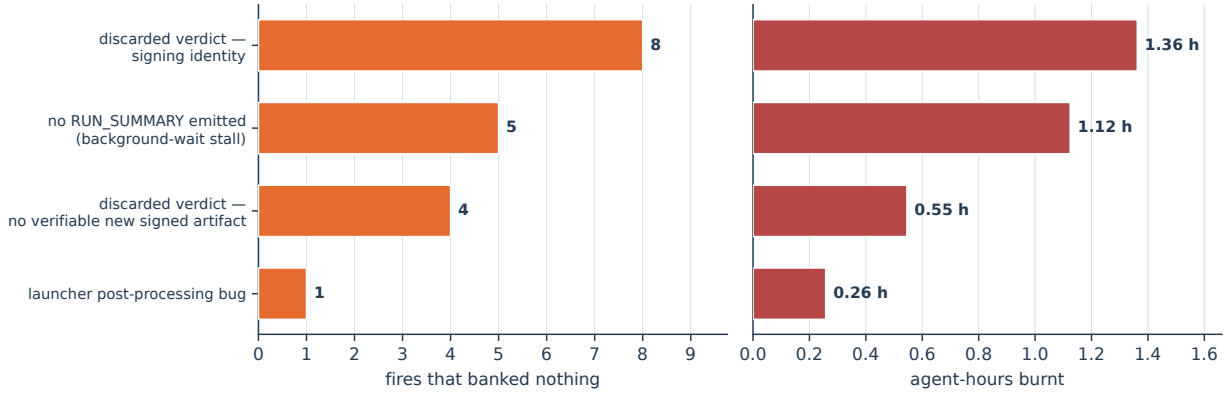
SECTION 4, CONTINUED

(e) Launcher post-processing and the evidence schema-flip guard

One fire (Aug 01 04:00, campaign item c7) did real work for 15 minutes and was then killed by the launcher's own post-processing: a `pipefail` interacting with a `grep` that legitimately matched nothing aborted the script before any record was written. It is the only fire in the set reconstructed by an operator after the fact, and it is the reason `signing_check` shows one `NOT_RUN`. Fixed in launcher v2.1.

Separately, an autocommit guard now watches for `RESTORED_SCHEMA_FLIP`: an agent rewriting a shipped evidence file into a different schema and the commit hook silently restoring the old one, which would leave the tree self-consistent and the history lying. No occurrence has been recorded since the guard shipped; the log needs watching rather than trusting.

What the defects cost



Cause	fires lost	agent-hours burnt
verdict discarded: signing identity	8	1.36
no RUN_SUMMARY emitted (background-wait stall)	5	1.12
verdict discarded: no verifiable new signed artifact	4	0.55
launcher post-processing bug	1	0.26
Total	**18**	**3.29**

18 of 209 fires (8.6%) banked nothing, costing 3.3 of the 33.6 agent-hours spent (9.8%). Every one of those hours is a fire whose work was real and whose bookkeeping was not, which is the correct way for this to fail: the loop discarded work it could not verify rather than recording work it could not verify.

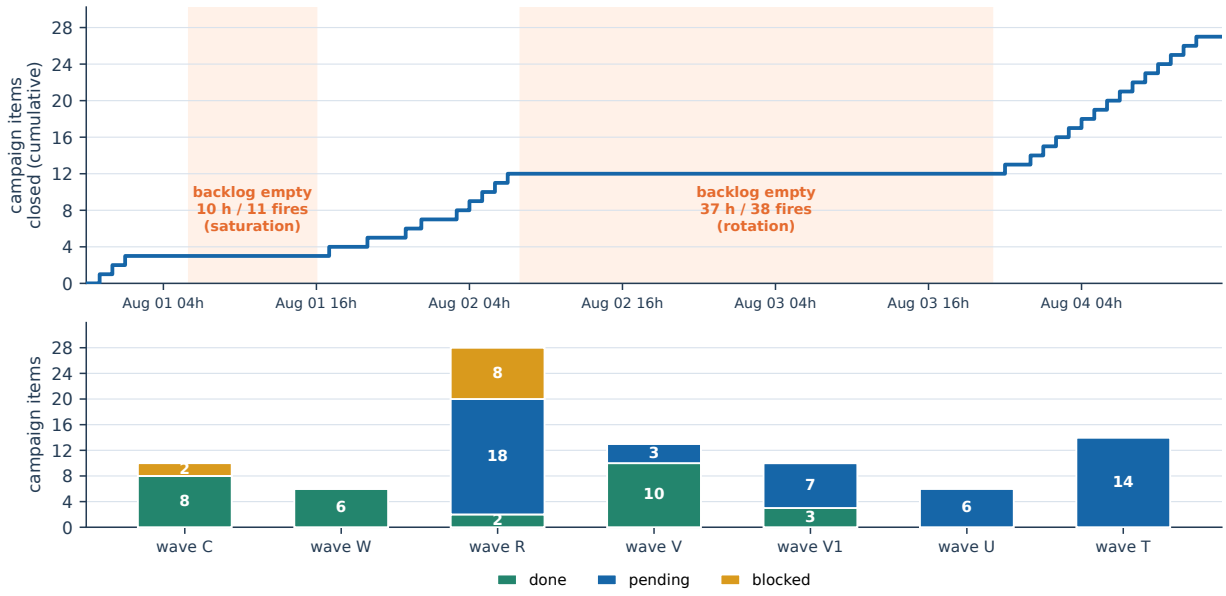
The proportion worth remembering. Five defects in the loop's own machinery cost 3.3 agent-hours out of 33.6. Over the same window the loop banked 78 signed verdicts and 43 repair receipts. A 10% overhead on a system that refuses to record work it cannot verify is a price worth paying — and in the current launcher generation it is down to 11% of fires, with the background-wait stall as the only remaining loss mode.

SECTION 5

Campaign mechanics and the backlog

The launcher owns `campaign.json` and the agent may never write it — the file is hash-guarded and a modification hard-stops the loop. Each fire the launcher takes the **first pending item** in the file and hands it over. That single rule is the whole scheduling policy, and it has one consequence worth stating to a non-operator: **item order is priority**. Re-ordering the file is how a human steers 209 hours of autonomous work, and it is the only steering input that does not require stopping the loop.

An item carries a `max_fires` budget and auto-blocks when it is spent, so a hard problem cannot eat the backlog. When nothing is pending, the launcher falls back to a stride-scheduled weighted rotation over the science modules — weight 2 for PRE_PHASE_0, weight 1 for already-validated PHASE_0.



The backlog-empty window

The largest gap ran from Aug 02 08:00 to Aug 03 21:00 — **37 hours** and 38 consecutive fires with an empty backlog, every one of them dispatched to undirected rotation. Earlier, an 10-hour window on Aug 01 produced 11 `SATURATED_SKIP` fires: not an empty backlog but a saturated one, every remaining item already at its `max_fires` ceiling. Those fires cost zero runtime — the launcher declined to spend the hour — which is the correct behaviour and the reason they are visually flat in the timeline.

Undirected rotation is not wasted: it produced 36 verdicts, including 8 survivals against claims nobody had chosen to defend. But its yield per hour is lower than directed work and it cannot make progress on a known defect. The window ended when Wave R was written into the backlog at Aug 03 22:00, and the survival rate moved within the same shift (section 2).

Backlog state at the pull

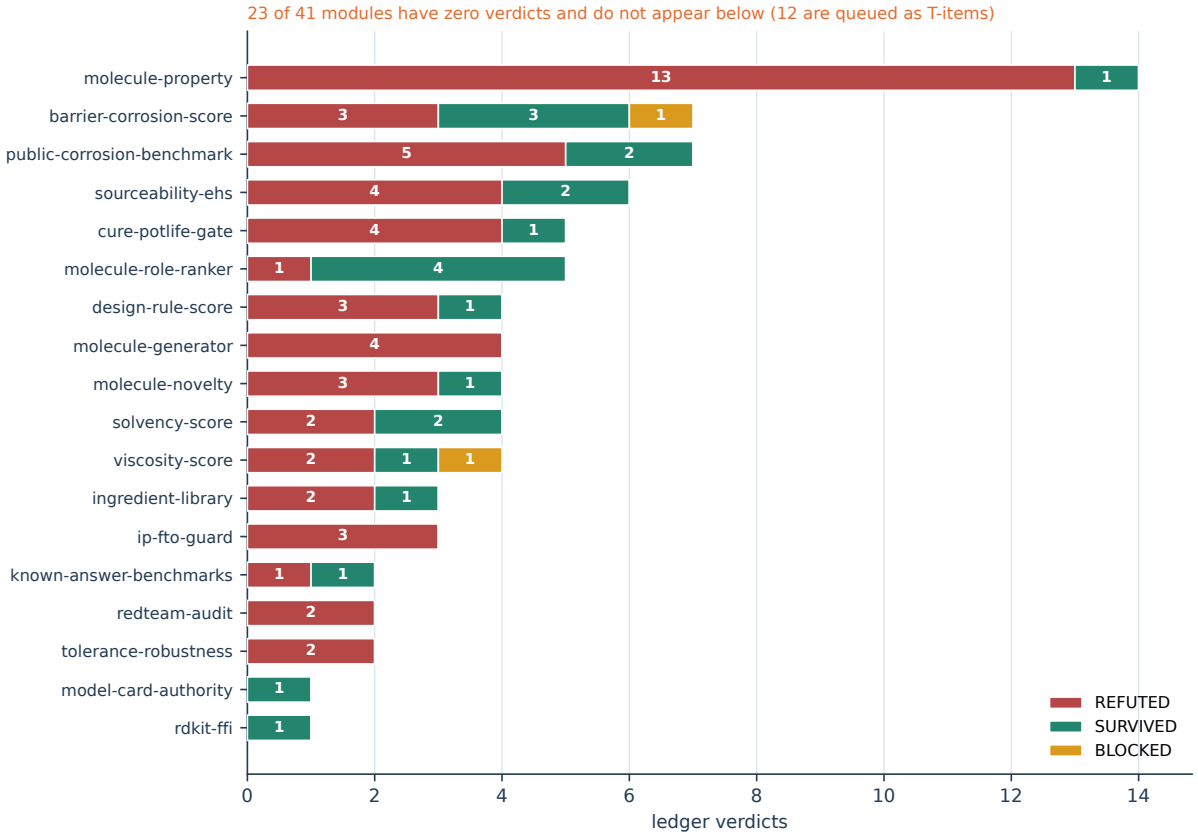
87 items: **29 done**, 48 pending, **10 blocked**. Every blocked item is blocked on a human, not on the loop — 10 of them are data-rights confirmations or portfolio-shape decisions the cron has no authority to make.

Wave	purpose	items	done	pending	blocked	fires
C	first falsification campaign	10	8	0	2	16
W	cross-corpus admissibility wave	6	6	0	0	8
R	repair wave, incl. the human-owned decisions	28	2	18	8	3
V	independent verify wave	13	10	3	0	10
V1	constructive strengthening wave	10	3	7	0	5
U	end-to-end / product-gap wave	6	0	6	0	0
T	first-attack wave for never-attacked modules	14	0	14	0	0

SECTION 6

Coverage: where the attacks actually landed

78 verdicts sounds like broad coverage. It is not evenly spread, and the uneven part is the honest finding of this section.



molecule-property alone carries 14 of the 78 verdicts (18%), because it was the subject of two multi-fire campaign items (C7, C8) on cross-corpus generalisation and then stayed in the rotation. At the other end, **23 of the 41 modules in the registry have zero ledger verdicts**.

The zero-verdict modules

12 of them are science or gate modules that have shipped signed receipts but have never had a claim adversarially attacked. Constructive receipt-building is not evidence of defensibility, so these are queued as Wave T (`T-01`–`T-14`), each with a module-specific probe rather than a generic sweep — the ip-fto-guard 8-alternate-encoding injection harness aimed at contract and claim-lock parsing, citation resolve-versus-support attacks on source-ledger, and so on:

> candidate-contract, cro-protocol-packager, envelope-gates, evidence-intake-validator, formulation-arithmetic, formulation-universe, molecule-design-gate, optimizer-orchestrator, process-qc-twin, report-packager, run-provenance, source-ledger

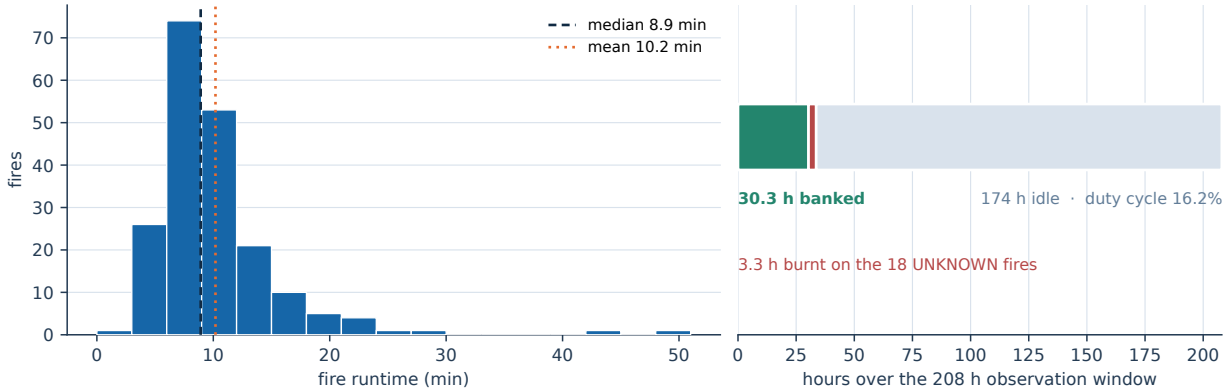
Two Wave T modules — redteam-audit and model-card-authority — have since picked up their first verdicts through waves R and V1, which is why the T backlog is 14 items but only 12 modules remain at zero.

The remaining 11 are infrastructure with no `mission.md` and therefore no scientific claim to attack (assurance-capabilities, bridge, cli, core, engine, formulation-capabilities, molecule-capabilities, ord-codec, phase0-reviewer, solver-lab, tui). They are correctly absent from the campaign; attacking them would generate paper, not information.

The coverage number that matters. Of the 41 registry modules, 18 have been adversarially tested and hold at least one signed verdict. Any statement about "the portfolio" that is not scoped to those 18 is a statement about untested code.

SECTION 7

What it cost



Measure	value
Wall-clock observation window	208 h
Total agent compute	33.62 h
Duty cycle (compute / wall)	16.2%
Fires	209
Mean fire runtime	10.2 min
Median fire runtime	8.9 min
90th-percentile fire runtime	15.7 min
Longest fire	48.4 min
Agent-hours burnt on 18 UNKNOWN fires	**3.29 h** (9.8% of compute)
Agent-hours per banked verdict (all compute)	0.43 h
Agent-hours per banked verdict (productive compute only)	0.39 h

The distribution is tight and right-skewed: half of all fires finish inside 9 minutes, 90% inside 16, and the tail is a handful of genuinely hard jobs — the longest, 48 minutes, is the pre-registered frozen analysis protocol written for barrier-corrosion-score. The hourly cadence is therefore not the binding constraint on throughput; 16% duty means the loop spends most of the wall clock idle by design, one fire per hour, no overlap. The lever for more work per day is a shorter interval or concurrent fires, not longer fires — and concurrency is exactly what the v3 `fire_id` hardening was built to make safe.

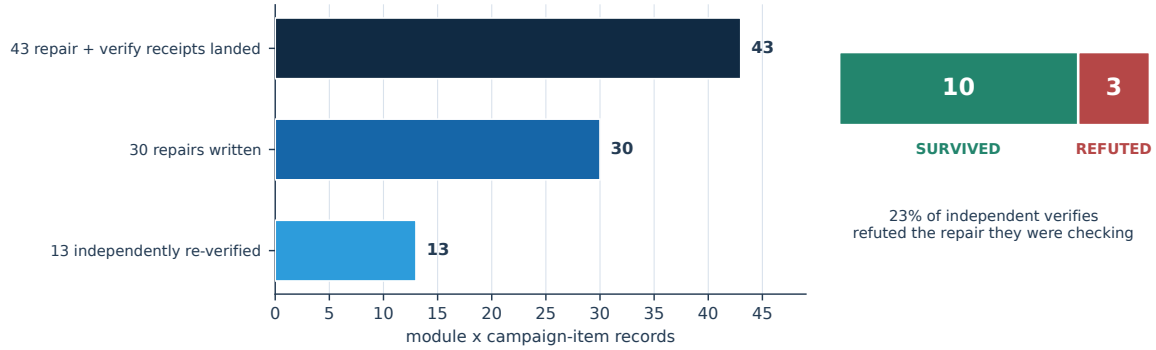
Three hours the loop never ran

The cadence should have produced one fire in each of the 209 hourly slots in the window (209 fires actually ran, three of them off-cadence catch-ups). 3 slots are empty: Jul 27 20:00, Jul 29 20:00, Jul 30 07:00 UTC. All three fall in the v1 custodial era, before the launcher recorded enough state to explain a miss, so the cause is not recoverable from the data — it is most likely host-level (the cron did not fire, or fired and died before writing). The gap is small and old, but it is unexplained, and an unexplained gap in a record that is supposed to be complete is worth naming.

SECTION 8

Repair, then arms-length verify

43 repair-and-verify receipts have landed. What makes them worth anything is the separation: an interactive session writes the repair, and a *later hourly fire, which did not write it and cannot see the reasoning that produced it*, is dispatched to attack it. The verify item's instruction is explicit — *"you did not write that repair; attack it"* — and it pre-commits the agent to CLAIM_REFUTED if no repair receipt exists at all, so a missing repair cannot be quietly skipped.



Verify item	module	verdict on the repair
V-A06	molecule-generator	REFUTED
V-A07	molecule-novelty	REFUTED
V1-RESOLVE	redteam-audit	REFUTED
V-A05	known-answer-benchmarks	SURVIVED
V-A08	public-corrosion-benchmark	SURVIVED
V-A12a	molecule-role-ranker	SURVIVED
V-A12b	sourceability-ehs	SURVIVED
V-N02a	molecule-property	SURVIVED
V-N02b	molecule-role-ranker	SURVIVED
V-N02c	rdkit-ffi	SURVIVED
V-N03	public-corrosion-benchmark	SURVIVED
V1-KPI-MCA	model-card-authority	SURVIVED
V1-SOLV	solvency-score	SURVIVED

The three that refuted their own repair

This is the strongest single piece of evidence in the report that verification is arms-length. 3 of 13 independent verifies came back **REFUTED**: the fire was sent to confirm a fix and instead found the fix wanting.

- `V-A06` / **molecule-generator** reproduced R-A06's arithmetic *exactly* — Clopper-Pearson intervals, IntDiv1 diversity, amine-site counts, all matching to four decimal places — and then refuted the repair anyway, on the *label convention*. R-A06's corrected headline of "29/38 difunctional" counts amine centres; but all 29 are `active_nh=4` primary diamines, which are tetrafunctional under the active-hydrogen convention this portfolio's own AHEW and Flory-Stockmayer receipts already use. Under its own house convention the true count is 0/38. An agent checking only the numbers would have passed it.
- `V-A07` / **molecule-novelty** found the fix correct but incompletely propagated. R-A07 applied the corrected IPDA structure ($C_9H_{20}N_2 \rightarrow C_{10}H_{22}N_2$) in a Morgan-only side table and left `maccs_166`, `atompair_2048` and `toptorsion_2048` on the refuted structure. Propagating it into all four fingerprint families flips one undisclosed cell from NOT-NOVEL to NOVEL, contradicting the receipt. One of four families fixed is not a fix.
- `V1-RESOLVE` / **redteam-audit** escalated R-A00's retraction-index sweep from "does the index exist and do its cited paths resolve" to "read each of the 80 rows across 16 module indexes and judge whether the cited receipt actually covers the retracted claim" — and refuted at the row level.

Both science refutations went straight back into work: `R-A06v2` and `R-A07v2` are second-generation repairs already landed against exactly those findings. That is the loop closing on itself — attack, repair, independent verify, refute the repair, repair again.

Why this is the credibility test. If independent verification always confirmed the repair, the correct conclusion would be that the verifier is not independent. A 23% refutation rate on repairs written by our own prior sessions is the number that says the check has teeth.

SECTION 9

Where these numbers come from

Not a single figure in this document was typed by hand. `scripts/loop_stats.py` parses the raw launcher records once; `scripts/gen_figures.py` and `scripts/gen_snippets.py` both import it, so a chart and the sentence next to it are computed from the same variable and cannot drift apart. Rebuild with `scripts/build.sh`; the `data/` directory travels with the document, so the PDF is reproducible from the tarball alone with no access to xbabe2.

The inputs

File	size	what it is
`ledger.jsonl`	78 rows	the append-only falsification ledger: one line per banked verdict
`fire_summary.jsonl`	209 rows	one line per fire, written by the launcher after the agent exits
`campaign.json`	87 items	the hash-guarded backlog; first pending item wins
`trust_chain.json`	444 receipts	per-receipt DSSE envelope / anchoring / verification state
`phase0_decision_report.json`	29 modules	the CLI verifier's derived phase state
`frontmatter.json`	41 modules	the declared phase state from each mission.md
`repairs_done.tsv`	43 rows	landed repair and verify receipts, module x campaign item
`modules.txt`	41 names	the module registry, the denominator for every coverage claim

All of it was pulled read-only at **2026-08-04T15:21Z** against master manifest `f5623b3171d65de2dc805b740aef71ad...`

Three parsing decisions a reader should be able to audit

1. **Corrupt timestamps are stripped, not dropped.** The 2 ledger rows carrying a leading backslash (section 4d) are parsed by removing the prefix. They are counted in every total in this report. Dropping them would have quietly reduced the verdict count from 78 to 76.
2. **Ledger order is file order.** The convergence exhibit indexes verdicts by their position in `ledger.jsonl`, not by timestamp, because append order is the property the file actually guarantees. The two happen to agree here — the timestamps are monotonic across all 78 rows — but where they could disagree, the append order is authoritative.
3. **Older fire records are missing fields, and absence is not zero.** Only 82 of 209 fires carry `signing_check` and only 64 carry `ledger_has_fire_id`, because the fields did not exist before the v2 and v3 launchers. Every rate in this report is quoted against the number of fires that could have produced the field, never against all 209. Era boundaries in section 3 are derived from exactly this: the first fire to emit a field is the first fire of the generation that invented it.

The one number this document cannot verify. Runtime is the launcher's wall-clock measurement of the agent subprocess, so it includes model latency, tool calls and filesystem work without separating them. The 33.6 agent-hours in section 7 is a true measure of how long the loop occupied the host — it is not a token-cost figure and should not be read as one.

What would falsify this report

This document makes four claims about the loop rather than about the science. Each has a cheap test, listed here so that a reader who does not trust the narrative can run one instead of arguing with it.

- **"Verification is arms-length."** Falsified if any verify receipt shares a fire id with the repair it checked, or if a verify's done receipt predates the repair receipt it cites. All 13 verifies in section 8 carry a distinct, later fire id.
- **"A downgraded fire banks nothing."** Partly false, and section 1 says so: joining `ledger.jsonl` to `fire_summary.jsonl` on `fire_id` finds 7 ledger lines belonging to fires the launcher downgraded. The downgrade denies the fire credit; it does not retract a line already appended. Falsified further if any of those 7 lines cites a receipt that fails signature verification — none do.
- **"Signing violations went to zero and stayed."** Falsified by any fire after Aug 03 21:00 with a non-zero `signing_violations`. The last 18 are all zero.
- **"Nothing here was typed by hand."** Falsified by any number in `snippets/` that is not reproduced by re-running `scripts/build.sh` against the shipped `data/`.

SECTION 10

Recommendations

Numbered, actionable, and ordered by what stops working soonest if ignored.

1. **Keep the backlog non-empty.** The single largest measured inefficiency in the run is the 37-hour window in which 38 fires ran undirected because nothing was pending. Waves T and U are already written and hold 20 pending items — enough for roughly 20 directed hours at the current cadence, plus 28 more in R/V/V1. The operational rule that follows from the first-pending-wins scheduler: refill before the pending count reaches single digits, and refill in priority order, because order *is* the priority.
2. **Run the `R-D07` re-evaluation ceremony.** This is the highest-value blocked item in the portfolio and it is blocked on a human, not on tooling. The governed repair path was built and tested this session (WV-1): `portfolio phase0 re-evaluate -module <M> -reason <text> -as-of <ts> -evaluation-key <external> -review <signed>`, which refuses to run unless the module is in the recorded unbound-benchmark state. Eight modules are repairable by it; recovery of the original evaluator bytes was proven impossible, so re-evaluation under dual identity is the only path back. Until it runs, the front-matter declares 17 modules at PHASE_0 and the CLI verifier derives 9.
3. **Decide `R-D08`: merge formulation-universe into formulation-arithmetic.** A portfolio-shape decision the cron has no authority to make. The module has zero ledger verdicts, carries no science of its own, and its claim is a strict subset of its sibling's. Merging loses no defensible claim and removes a module from every portfolio count — which is the point: a smaller honest denominator beats a larger one padded with untested modules.
4. **Close the two live `V1-KPI-QC` stalls before the item auto-blocks.** Both of the last two fires produced no `RUN_SUMMARY`; the item has one fire of budget left. Either the prompt-v3.1 background-wait prohibition is not reaching this item, or the job genuinely does not fit in an hour and should be split. Diagnose before spending the last fire.
5. **Account for the 3 missing fire hours** (Jul 27 20:00, Jul 29 20:00, Jul 30 07:00 UTC). All three predate the launcher's state recording, so the cause cannot be recovered from the data. Add a cron-side heartbeat that records an attempted fire even when the agent never starts, so that a future gap is explained rather than merely observed.
6. **Tell downstream consumers to tolerate the 2 corrupt ledger timestamps.** They cannot be repaired: the falsification ledger is append-only and rewriting it to fix a cosmetic escape would cost more credibility than the defect does. Any parser that reads `logs/falsification-ledger.jsonl` must strip a leading backslash from `ts` before parsing — including the ones we have not written yet. The generating defect is fixed; the residue is permanent.
7. **Watch the autocommit guard log for `RESTORED_SCHEMA_FLIP`.** No occurrence has been recorded since the guard shipped. That is a reason to keep reading the log, not to stop: a silent schema restore is the one failure mode in this system that would leave the tree self-consistent and the history wrong, and it would not show up in any of the three verification checks in section 1.

The standing invariant

Everything above is maintenance on one property: **a verdict the launcher cannot independently verify does not count**. 12 verdicts were discarded under that rule in this window, at a cost of 3.3 agent-hours. The rule is not negotiable, because the alternative — trusting the agent's own report of its own work — is the failure mode the entire loop exists to prevent.

Data pulled read-only 2026-08-04T15:21Z. Figures and every number in this document are generated by `scripts/gen_figures.py` and `scripts/gen_snippets.py` from `data/`; no value in this PDF was typed by hand. Trust chain at the pull: 186/258 loop-era receipts verify against the pinned key, 0/186 pre-loop.